

Developing a Program to Create RFC 9883 Compliant Certificate Signing Requests

Nolan Attreed

ntattreed@vt.edu

Virginia Polytechnic Institute and State University
Alexandria, Virginia, USA

Abstract

A key component of the Transport Layer Security protocol is the Digital Certificates used to establish the Public Key Infrastructure (PKI). These certificates are signed with cryptographic keys by intermediary Certificate Authorities who in turn have their certificates signed by the trusted Root Authorities. The validation of the signature from leaf node certificates up to the Root Authority's certificates establishes what is known as the Chain of Trust. However, the rise of Post Quantum Computing introduces adversarial mechanisms like Shor's algorithm, which undermines the primitive cryptographic algorithms used in PKI. As such, the National Institute of Standards and Technology presented cryptographic algorithms meant to replace them and are quantum safe. However, this requires modifications to the PKI used today as ML-KEM, a Key Encapsulation Mechanism, does not provide a way to generate Certificate Signing Requests for its public key. RFC 9883 introduces a Private Key Statement of Possession attribute to add into PKI allowing a requesting entity to sign with an alternative signing key. In this paper, two programs are provided to implement RFC 9883 compliant Certificate Signing Requests and use them to build a ML-KEM digital certificate. In doing so, experiments find that a full Post Quantum chain of trust contains more than double the amount of overhead for its components.

Keywords

Post Quantum Cryptography, ML-KEM, ML-DSA, Digital Certificates, Chain of Trust, RFC 9883, OpenSSL, Certificate Signing Requests

ACM Reference Format:

Nolan Attreed. 2026. Developing a Program to Create RFC 9883 Compliant Certificate Signing Requests. In *Proceedings of System and Software Security (ECE 5590)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Academic Integrity Pledge

I wrote this report myself, not using generative AI technologies.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ECE 5590, Alexandria, VA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/05
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Digital Certificates continue to remain a fundamental component of the modern internet. Housed within them are the identifying information and public key for a given end entity. When a client visits a website, often this certificate is exchanged so that the user can verify and use the public key to exchange a shared secret in a confidential manner with a server. In doing so, they create a secure channel for end-to-end encryption. The verification for these certificates comes in the form of digital signatures, where certificates contain signatures that can be verified by the public key matching the private key that signed it [4]. This signature is performed by a certificate authority, which in turn has its own signing key signed by a root authority. Root certificates are typically self-signed, installed on end-user devices so that when presented a leaf certificate from a webserver, they can fetch all intermediate certificates and verify the signatures present. This is what creates the chain of trust in the Public Key Infrastructure (PKI).

The cryptographic keys used in the modern PKI system today include asymmetric primitives such as RSA and Elliptic Curve Cryptography. However, the rapid evolution of quantum computing poses a threat to these schemes. Developed in 1994, Shor's Algorithm revealed that with quantum computers one can factor large numbers quickly [2]. This undermines the core security property of algorithms like RSA in regard to factorization of large prime numbers. As such, in 2013, the National Institute of Standards and Technology (NIST) launched the Post Quantum Cryptography (PQC) competition to identify and standardize algorithms that would be resistant to the attacks presented by quantum computers [9]. Among the algorithms standardized were the module lattice-based schemes ML-KEM[11], for shared secret derivation, and ML-DSA[10], for digital signatures, developed around the Module Lattice Learning-With-Errors problem [7].

In early 2025, a presentation was given by NIST regarding the transition to PQC, with algorithms like RSA to be deprecated by 2030 [8]. PKI architecture has begun to transition into the usage of PQC algorithms, though mainly in a hybrid state. Open-source tools, such as OpenSSL, have provided support for these algorithms including their usage in digital certificates as of version 3.5+. However, neither ML-KEM nor ML-DSA can be used as a direct stand-in for RSA/ECC in leaf nodes. With RSA/ECC, these leaf entities can use their keys for asymmetric encryption, but they can also perform signatures for components like Certificate Signing Requests (CSR's). ML-KEM can encrypt shared secrets but cannot self-sign to be used in a CSR to pass along to a CA for a signed certificate. ML-DSA can self-sign to be used in a CSR but is unable to encrypt a shared secret. This paper explores two programs: `generateCSR`,

Component	RSA 2048	RSA 4096
Root Certificate	1.24 KB	1.94 KB
CA Certificate	1.28 KB	1.97 KB
CA CSR	1.03 KB	1.72 KB
Leaf Certificate	1.21 KB	1.90 KB
Leaf CSR	956 B	1.62 KB
Private Key File	1.69 KB	3.24 KB
Public Key File	460 B	814 B

Table 1: RSA Certificate Chain Components

a program developed using OpenSSL to implement RFC 9883 compliant CSRs [6], providing a mechanism for creating a request for ML-KEM keys to be signed by a CA leveraging ML-DSA keys, and generateCertificate, which takes this new CSR, validates the signature and creates a certificate for the ML-KEM key. In doing so, it is shown that creating a fully PQC certificate chain is achievable with the trade off of increased component overhead.

2 Related Work

Before diving into a full PQC certificate chain, it is worth exploring what components make up a classical chain and the respective overhead introduced by each component. Listing 4 in Appendix A.1 shows the sequence of commands that can be executed to create a fictional certificate chain for a root CA, intermediary CA and leaf node. Table 1 shows the components and their sizes for both the standard 112-bit security and the maximum 192-bit security. Most components are manageable in size, remaining under 2 KB in size. The largest components are the private key files which also contain the public keys encoded within them as well.

D’Onghia et al. presented recommendations for transitioning into a PQC safe PKI, including detailing what components of the X.509 certificate structure would have to change to utilize PQC schemes [5]. In their paper, it is recommended that the alternative fields of a certificate are used to hold the new key information. However, OpenSSL’s integration of the schemes uses the regular fields to maintain full compatibility and provide a more seamless plugin. Also noted in the work by D’Onghia et al. are the varying key sizes and signature sizes for the many PQC algorithms, including the ones used in this project. Using a full ML-DSA and ML-KEM certificate chain will likely introduce significant overhead to all components of the certificate chain.

3 Approach

Listing 1: Private Key Possession Statement Attribute

```
id-at-statementOfPossession OBJECT IDENTIFIER
 ::=
 { 1 3 6 1 4 1 22112 2 1 }

privateKeyPossessionStatement ATTRIBUTE ::= {
  TYPE PrivateKeyPossessionStatement
  IDENTIFIED BY id-at-statementOfPossession }

PrivateKeyPossessionStatement ::= SEQUENCE {
  signer IssuerAndSerialNumber,
```

```
cert Certificate OPTIONAL }
```

For providing a strategy for ML-KEM certificate signing requests RFC 9883 proposed a new attribute for usage in CSRs. This attribute, named a privateKeyPossessionStatement, which will allow requesting entities to sign their CSRs with another signing key pair. This alternative will allow the entity to sign their certificate and provide a statement in the CSR that tells the CA you gave the matching private key to the ML-KEM pair, providing non-repudiation. Listing 1 presents the attribute structure which contains the issuer (i.e. CA) and serial number of the entities signing key certificate, as well as optionally the signing keys certificate itself. By providing the certificate, the signing authority does not need to fetch the public key required to validate the CSR signature. Opting out of including this certificate assumes the CSR is being signed by the same CA and that the CA contains a methodology for fetching certificates they signed. OpenSSL provides a simple method for creating new ASN1 attributes, which is leveraged in the implementation of generateCSR and generateCertificate.

4 Threat Model

Figure 1 details the threat model that a PQC resistant PKI would be preventing. In this scenario, the adversary collects a benign web servers’ certificate (1,2), which contains their public key, for this example we will say it is an RSA key. With access to a capable quantum computer, the adversary can then use Shor’s algorithm to factor the prime numbers used for the RSA public key (3) and in doing so can compute the secret key belonging to the benign site. The attacker could then use this compromised key pair for malicious purposes, like a man-in-the-middle (MITM) server (4). Now a benign user might be redirected to the MITM server and when presented with the valid certificate, would unknowingly trust the malicious server, breaking end-to-end encryption (5,6).

5 Implementation

Listing 2: Private Key Possession Statement Attribute

```
1 #define SOP_OID_STR "1.3.6.1.4.1.22112.2.1"
2 #define SOP_SN      "id-at-statementOfPossession"
3 #define SOP_LN      "privateKeyPossessionStatement"
4
5 typedef struct
6 {
7     X509_NAME      *issuer;
8     ASN1_INTEGER   *serialNumber;
9 } ISSUER_AND_SERIAL;
10
11 ASN1_SEQUENCE (ISSUER_AND_SERIAL) = {
12     ASN1_SIMPLE (ISSUER_AND_SERIAL, issuer,
13                 X509_NAME),
14     ASN1_SIMPLE (ISSUER_AND_SERIAL, serialNumber,
15                 ASN1_INTEGER)
16 } ASN1_SEQUENCE_END (ISSUER_AND_SERIAL)
17
18 IMPLEMENT_ASN1_FUNCTIONS (ISSUER_AND_SERIAL)
19
20 typedef struct {
21     ISSUER_AND_SERIAL *signer;
```

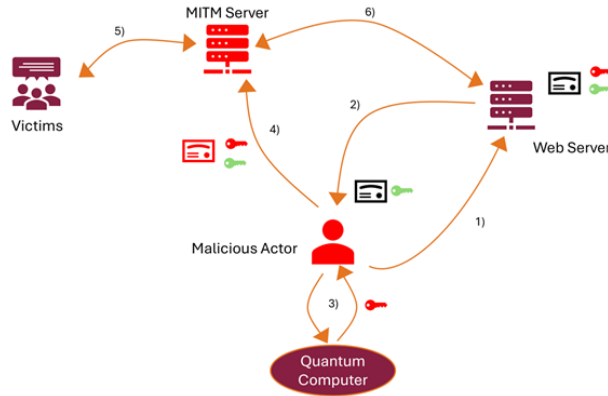


Figure 1: Compromised Key Model

```

20 X509 *cert;
21 } PRIVATE_KEY_POSSESSION_STMT;
22
23 ASN1_SEQUENCE(PRIVATE_KEY_POSSESSION_STMT) = {
24     ASN1_SIMPLE(PRIVATE_KEY_POSSESSION_STMT,
25         signer, ISSUER_AND_SERIAL),
26     ASN1_OPT(PRIVATE_KEY_POSSESSION_STMT, cert,
27         X509)
28 } ASN1_SEQUENCE_END(PRIVATE_KEY_POSSESSION_STMT)
29
30 IMPLEMENT_ASN1_FUNCTIONS(
31     PRIVATE_KEY_POSSESSION_STMT)

```

5.1 ASN.1 Attribute Template

OpenSSL provides a template to create new attributes defined through `asn1t.h` [12]. Listing 2 shows the C implementation of this template scheme used to create the key possession statement attribute. To represent the issuer and serial number, a struct is used to package them together. `ASN1_SEQUENCE` and `ASN1_SIMPLE` are used to then declare the ASN1 sequence so that it can be properly encoded / decoded into the ASN.1 based X.509 certificate. `IMPLEMENT_ASN1_FUNCTIONS` dynamically creates the functions necessary to allocate a new attribute, free its components, create encoded strings, etc. This process is then repeated to embed the Issuer and Serial number into a private key attribute, which also contains the certificate. Using this simple method, the program can now use basic OpenSSL API calls to construct RFC 9883 compliant CSRs as well as break down a CSR and build a signed certificate.

Listing 3: Registering Possession Statement OID

```

1 #define SOP_OID_STR "1.3.6.1.4.1.22112.2.1"
2 #define SOP_SN      "RFC_9883"
3 #define SOP_LN      "privateKeyPossessionStatement"
4
5 static int register_sop_oid(void){
6     int nid = OBJ_txt2nid(SOP_OID_STR);
7     if (nid == NID_undef) {
8         nid = OBJ_create(SOP_OID_STR, SOP_SN,
9             SOP_LN);

```

```

9     if (nid == NID_undef){
10         handleSSLerr("Could not register OID
11             : " SOP_OID_STR);
12     }
13     return nid;
14 }

```

Listing 3 details the function responsible for registering the object's numerical identifier from a short name, a long name and the object identifier defined from RFC 9883 [3]. The `OBJ_txt2nid` returns a special value if the OID is not present in OpenSSL's internal table. This allows for a check to ensure a duplicate NID is created for the statement of possession. Should the program indicate that it is not registered, `OBJ_create` is used to define the NID for later usage in API calls.

5.2 Reading PEM Files

Various files need to be passed to the programs to correctly build both the 9883 compliant CSR and the certificate from said CSR. This can be achieved through the use of the `bio.h` header from the OpenSSL source code [12]. This header comes equipped with a simple API not only for reading in the PEM files themselves but automatically allocating and storing the file contents in their respective OpenSSL data types. For CSR generation this includes the reading of an ML-KEM public key and ML-DSA private signing key for storage in `EVP_PKEY` objects, and the ML-DSA issued certificate for storage in an `X509` object. Likewise for the generation of a new certificate this requires reading of the generated CSR and storing its contents in the `X509_REQ` data object, reading the CA's certificate into a `X509` object and the signing key into an `EVP_PKEY`.

5.3 Generating an RFC 9883 CSR

Generation of a new CSR is simple with OpenSSL's expansive API library. The statement of possession NID is registered as a first step so that necessary helper functions can be used. All additional components mentioned in Section 5.2 are also loaded. An `X509_REQ` object is then allocated for the program to manipulate, and in a similar fashion to the actual CSR generating OpenSSL command,

the user is prompted for subject information. This includes information standard to the X.509 certificate like an entities country of origin, Fully Qualified Domain Name (FQDN) and other identifying information. Once collected and added into the CSR, the statement of possession attribute is then created.

From Section 5.1, the `PRIVATE_KEY_POSSESSION_STMT_new()` function is created and allows for simple allocation of a statement. Following this, the passed certificate for the end user’s signing key is used to extract the issuer and serial number information, and the certificate itself is embedded into the statement object. Additionally, the `i2d_PRIVATE_KEY_POSSESSION_STMT()` function is created for simple encoding of the attribute into an DER encoded ASN.1 sequence. Since the NID was also registered in step 1, OpenSSL’s API call for adding an attribute by its NID can be leveraged to do the manipulation and storage of the encoded statement. Finally, the end entity’s private ML-DSA key is used to sign the CSR and write its contents out to a PEM file. All data objects are also required to manually be freed using their API calls. Now, a usable CSR file that is RFC 9883 compliant can be leveraged by an end entity.

5.4 Generating a Certificate from an RFC 9883 CSR

Creating a certificate from an RFC 9883 CSR is also simple using OpenSSL’s API. As conducted in the creation of a CSR, the NID is registered for usage and necessary components are fetched from their files. The key possession statement is then extracted using a combination of functions. `X509_REQ_get_attr_by_NID` uses the NID to fetch the index, if present, of the attribute object within the entire X509 certificate. Upon retrieval of this index, `X509_REQ_get_attr` can fetch the attribute using the index and `X509_ATTRIBUTE_get0_type` obtains the actual encoded string. Like in Section 5.3, `d2i_PRIVATE_KEY_POSSESSION_STMT` is created and can be used to take the DER encoded string converting it back into the object created by the code in Section 5.1. Using this attribute, the program can fetch the public key and validate the signature on the CSR. Once fetched, standard OpenSSL calls can be used to gather the rest of the information needed to build and write the final certificate ML-KEM certificate.

6 Evaluation

The code for this report can be found at [1]. Using the commands in Appendix A.2 Listing 5, a full PQC chain can be developed to verify the usage of this new attribute. The chain results in a successful verification with the final command showing that the chain of trust is upheld. Table 2 shows the full certificate chain contents and their sizes. What is immediately noticeable is the overhead. ML-KEM and ML-DSA contain both large key sizes and in ML-DSA’s case a large signature. But, ML-DSA is also designed to have a faster signature, so this tradeoff is acceptable.

Because of this, the CSRs now exceed 10 KBs in size and close in near 20 KB for maximum security with the two algorithms. However, these CSRs are temporary until the end entity obtains their certificate. The certificate itself is less than half the size for the ML-KEM key. The work does confirm, however, that certificate authorities need to be prepared for the increased overhead PQC chains will bring with them. It was recommended in the work from

Chain Component	KEM-512/DSA-44	KEM-768/DSA-65	KEM-1024/DSA-87
Root Certificate	5.55 KB	7.60 KB	10.2 KB
CA CSR	5.33 KB	7.39 KB	10.0 KB
CA Certificate	5.58 KB	7.64 KB	10.2 KB
Leaf DSA Certificate	5.51 KB	7.57 KB	10.1 KB
Leaf DSA CSR	5.27 KB	7.32 KB	9.95 KB
Leaf KEM Certificate	4.84 KB	6.54 KB	8.83 KB
Leaf KEM CSR	10.2 KB	13.9 KB	18.9 KB
DSA Key File	3.58 KB	5.55 KB	6.71 KB
KEM Private Key File	2.38 KB	3.41 KB	4.44 KB
KEM Public Key File	1.15 KB	1.67 KB	2.18 KB

Table 2: PQC Certificate Chain Component Size

[5] that root certificates maintain the highest security levels. As such, while making a simple adjustment, client machines need to make sure enough space is allocated for storage of these certificates. On lightweight devices, however, this could come at a significant cost.

7 Conclusion

The work in this paper introduced a C implementation of the attribute proposed by RFC 9883, providing a methodology for producing certificate signing requests for an ML-KEM public keys. In doing so, a user can use an alternative signing key to sign the CSR, storing the signing key information as a proof of private key statement. Furthermore, a program was developed to take such a CSR, validate its signature and create a certificate using a CA signing key. OpenSSL provided a simple API for the development of the attribute and providing simple methods to develop usable PEM files for the components introduced in this paper.

While the full transition to Post Quantum Cryptography within the Public Key Infrastructure may see alternative methods used, this at least provides a usable proof of concept for a lab environment. Indeed, further investigation is worthwhile to explore how the overhead of these certificates and signing requests impact the latency for full TLS handshakes in a web environment, or how much of a size impact this has on highly constrained devices. With this overhead in mind, both CAs and Root CAs need to consider the impact larger certificates will introduce in their own environments, especially as higher security levels are desired for their signing keys.

References

- [1] Nolan Attreed. [n. d.]. *RFC_9883_POC*. https://github.com/nattreed/RFC_9883_POC/tree/main
- [2] Classiq. 2024. *Shor’s Algorithm*. Retrieved May 09, 2025 from <https://www.classiq.io/algorithms/shors-algorithm>
- [3] die.net. [n. d.]. *obj_txt2nid(3) - Linux man page*. Retrieved May 09, 2025 from https://linux.die.net/man/3/obj_txt2nid

- [4] DigiCert. 2023. *How Certificate Chains Work*. Retrieved May 09, 2025 from <http://math.tntech.edu/rafal/cliff11/index.html>
- [5] Grazia D'Onghia, Diana Gratiela Berbecaru, and Antonio Lioy. 2026. Shaping a Quantum-Resistant Future: Strategies for Post-Quantum PKI. arXiv:2601.11104 [cs.CR] doi:10.48550/arXiv.2601.11104
- [6] Russ Housley. 2025. An Attribute for Statement of Possession of a Private Key. RFC 9883. <https://www.rfc-editor.org/rfc/rfc9883.html>
- [7] Adeline Langlois and Damien Stehle. 2012. Worst-Case to Average-Case Reductions for Module Lattices. Cryptology ePrint Archive, Paper 2012/090. Retrieved May 09, 2025 from <https://eprint.iacr.org/2012/090>
- [8] Dustin Moody. 2025. *NIST PQC The Road Ahead*. Retrieved May 09, 2025 from <https://csrc.nist.gov/csrc/media/Presentations/2025/nist-pqc-the-road-ahead/images-media/rwcpqc-march2025-moody.pdf>
- [9] National Institute of Standards and Technology. 2017. *Post-Quantum Cryptography*. Retrieved May 09, 2025 from <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [10] National Institute of Standards and Technology. 2024. Module-Lattice-Based Digital Signature Standard (ML-DSA). FIPS PUB 204. doi:10.6028/NIST.FIPS.204
- [11] National Institute of Standards and Technology. 2024. Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM). FIPS PUB 203. doi:10.6028/NIST.FIPS.203
- [12] OpenSSL Project. 2025. OpenSSL 3.5 Source Repository. [https://github.com/openssl/openssl-3.5](https://github.com/openssl/openssl/tree/openssl-3.5). GitHub repository.

A OpenSSL Methods

A.1 RSA Certificate Chain

Listing 4: Classical Certificate Chain Example

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:BITSIZE
openssl pkey -in private_key.pem -pubout -out public_key.pem

openssl genpkey -algorithm RSA -out root_key.pem -pkeyopt rsa_keygen_bits:BITSIZE
openssl req -new -x509 -key root_key.pem -out root_cert.pem -days 3650 -subj "/C=US/O=Fictional Root Authority/CN=Fictional Root CA"

openssl genpkey -algorithm RSA -out ca_key.pem -pkeyopt rsa_keygen_bits:BITSIZE
openssl req -new -key ca_key.pem -out ca_csr.pem -subj "/C=US/O=Fictional Intermediate Authority/CN=Fictional RSA CA" -addext "basicConstraints=critical,CA:TRUE" -addext "keyUsage=critical,keyCertSign,cRLSign"
openssl x509 -req -in ca_csr.pem -CA root_cert.pem -CAkey root_key.pem -CAcreateserial -out ca_cert.pem -days 1825 -copy_extensions copy

openssl req -new -key private_key.pem -out entity_csr.pem -subj "/C=US/O=Example Org/CN=example.com"
openssl x509 -req -in entity_csr.pem -CA ca_cert.pem -CAkey ca_key.pem -CAcreateserial -out entity_cert.pem -days 365

openssl verify -CAfile root_cert.pem -untrusted ca_cert.pem entity_cert.pem
```

A.2 ML-KEM/DSA Certificate Chain

Listing 5: PQC Certificate Chain Example

```
openssl genpkey -algorithm ML-DSA-LEVEL -out mldsa_key.pem
openssl genpkey -algorithm ML-KEM-LEVEL -out mlkem_key.pem
openssl pkey -in mlkem_key.pem -pubout -out mlkem_pub.pem

openssl genpkey -algorithm ML-DSA-LEVEL -out root_key.pem
openssl req -new -x509 -key root_key.pem -out root_cert.pem -days 3650 -subj "/C=US/O=Fictional Root Authority/CN=Fictional Root CA"

openssl genpkey -algorithm ML-DSA-LEVEL -out ca_key.pem
openssl req -new -key ca_key.pem -out ca_csr.pem -subj "/C=US/O=Fictional Intermediate Authority/CN=Fictional ML-DSA CA" -addext "basicConstraints=critical,CA:TRUE" -addext "keyUsage=critical,keyCertSign,cRLSign"
openssl x509 -req -in ca_csr.pem -CA root_cert.pem -CAkey root_key.pem -CAcreateserial -out ca_cert.pem -days 1825 -copy_extensions copy

openssl req -new -key mldsa_key.pem -out entity_sign_csr.pem -subj "/C=US/O=Example Org/CN=example.com" -addext "keyUsage=digitalSignature"
openssl x509 -req -in entity_sign_csr.pem -CA ca_cert.pem -CAkey ca_key.pem -CAcreateserial -out entity_sign_cert.pem -days 365

openssl verify -CAfile root_cert.pem -untrusted ca_cert.pem entity_sign_cert.pem
..\path\to\repo\run\generate_csr.exe mlkem_pub.pem mldsa_key.pem entity_kem_csr.pem entity_sign_cert.pem
..\path\to\repo\run\generate_cert.exe entity_kem_csr.pem ca_cert.pem ca_key.pem entity_kem_cert.pem
openssl verify -CAfile root_cert.pem -untrusted ca_cert.pem entity_kem_cert.pem
```